Manual técnico — Pac-Man Multijugador

Universidad: Universidad Latina de Costa Rica (Sede: Guapiles)

Curso: Programación V

Integrantes:

- José Pablo Matarrita Quirós
- Sheymar Antonio Martínez Francis
- Mauricio castillo López

Profesor:

- Osvaldo Fabian Granados Venegas

Contenido

1. Descripción general del sistema 5
1.1 Componentes principales 5
1.2 Personajes y bandos 5
1.3 Modos de juego 5
1.4 Reglas del juego 6
1.5 Objetos 6
1.6 Configuración de la sala 6
1.7 Funciones de la aplicación 7
2. Arquitectura 8
2.1 Capas de la solución 8
2.2 Patrones de diseño 8
2.3 Flujo de una petición REST 9
2.4 Flujo de una jugada en tiempo real 9
3. Tecnologías, paquetes y herramientas 10
3.1 Backend (.NET) 10
3.2 Cliente (npm) 10
3.3 Android 11
3.4 Base de datos 11
3.5 Herramientas de desarrollo 11
4. Estructura del repositorio 12
4.1 Convenciones de código 12
5. Base de datos 13
5.1 Scripts y orden de ejecución 13
5.2 Tablas 13
5.3 Relaciones 14
5.4 Índices 14
5.5 Vistas 15
5.6 Niveles y formato del laberinto 15
6. Backend: PacmanApi 16
6.1 Registro de servicios (Program.cs) 16
6.2 Endpoints REST 16
6.3 Hub de tiempo real (SignalR) 17
6.4 Autenticación 18

6.5 Correo electrónico 18
6.6 Página /conectar 19
6.7 CORS 19
7. Motor del juego 20
7.1 Orden de un tick 20
7.2 Sistemas 20
7.3 Velocidades y tiempos 20
7.4 Puntaje 21
7.5 Inteligencia artificial 21
7.6 Objetos y efectos 21
7.7 Modelo del dominio del juego 21
8. Gestor de salas 23
8.1 Estados de una sala 23
8.2 Inicio y persistencia 23
8.3 Asignación de personajes 23
8.4 Ciclo, concurrencia y conexiones 23
9. Cliente: Phaser y TypeScript 25
9.1 Escenas 25
9.2 Servicios 25
9.3 Lógica del cliente 26
9.4 Presentación 26
9.5 Pantalla de juego 26
9.6 Pantalla adaptable 26
10. Seguridad 27
10.1 Contraseñas 27
10.2 Códigos por correo 27
10.3 Sesiones 27
10.4 Juego 27
10.5 Configuración y secretos 27
11. Instalación y configuración 28
11.1 Requisitos 28
11.2 Base de datos 28
11.3 Configuración de la API 28
11.4 Cliente web 28
12. Aplicación Android 30

12.1 Generar la APK 30
12.2 Conectar el celular al servidor 30
13. Solución de problemas 31
14. Anexos 32
14.1 Glosario 32
14.2 Constantes principales 32
14.3 Referencias 32

1. Descripción general del sistema

Pac-Man Multijugador es un videojuego de laberinto inspirado en el género de los juegos de persecución tipo arcade, pensado para jugarse en grupo desde celulares Android o desde el navegador. Hasta cuatro personas comparten una misma partida: unas controlan a los Pac-Man, que deben comerse todas las bolas del laberinto, y otras controlan a los fantasmas, que intentan atraparlos. Los personajes que no tienen jugador humano los mueve una inteligencia artificial (IA) que corre en el servidor.

El documento describe cómo está construido el sistema, qué tecnologías usa, cómo se organiza el código, cómo se guarda la información y qué pasos se siguen para instalarlo, ejecutarlo y generar la aplicación de Android. Está dirigido a desarrolladores, docentes evaluadores y cualquier persona que necesite mantener o ampliar el proyecto.

1.1 Componentes principales

El sistema tiene tres piezas que trabajan juntas:

- **Servidor (PacmanApi):** aplicación ASP.NET Core en C# que expone una API REST para cuentas, catálogos, historial y ranking, y un hub de SignalR para el juego en tiempo real. Aquí se simula toda la partida: movimiento, colisiones, puntos, objetos e IA.
- **Base de datos (PacmanDB):** SQL Server guarda usuarios, sesiones, catálogos (personajes, dificultades, niveles y objetos) y el resultado de cada partida, ronda y participante. El historial y los rankings se calculan con vistas.
- **Cliente (Pacman):** juego hecho con Phaser 4 y TypeScript. Dibuja lo que el servidor le envía, interpola el movimiento para que se vea fluido y manda al servidor las direcciones y el uso de objetos. Se ejecuta en el navegador o empaquetado como APK de Android con Capacitor.

> El cliente nunca calcula reglas. Solo envía intenciones (dirección, usar objeto) y dibuja los estados que recibe. Esto evita trampas y asegura que todos los jugadores vean la misma partida.

1.2 Personajes y bandos

Id	Personaje	Bando	Color	Comportamiento de la IA
1	Pac-Man 1	Pac-Man	Amarillo	Come bolas, huye de fantasmas cercanos y persigue a los asustados
2	Pac-Man 2	Pac-Man	Verde	Igual que Pac-Man 1 (solo existe en multijugador)
3	Blinky	Fantasma	Rojo	Persigue directamente al Pac-Man más cercano
4	Pinky	Fantasma	Rosa	Embosca: apunta 4 casillas por delante del Pac-Man
5	Inky	Fantasma	Celeste	Encierra junto con Blinky (duplica el vector desde Blinky)

1.3 Modos de juego

- **Contra la IA (1 jugador):** el jugador elige ser Pac-Man o fantasma (o al azar). Hay un solo Pac-Man (el personaje 1) y tres fantasmas; todo lo demás lo controla la IA. El rol elegido se conserva en todos los niveles.
- **Multijugador (2 a 4 jugadores):** los humanos se sortean entre los personajes 1 a 4 y el resto (siempre incluido Inky, el personaje 5) lo controla la IA. Como máximo hay 2 Pac-Man humanos y 2 fantasmas humanos.

Una partida es una serie de **5 niveles** (laberintos distintos). Entre nivel y nivel hay una transición de 5 segundos en la que se muestran los resultados. Si un jugador pierde la conexión conserva su lugar y la IA mueve su personaje hasta que vuelva.

1.4 Reglas del juego

- **Bolas:** cada bola normal vale 10 puntos y cada bola de poder 50. La bola de poder asusta a los fantasmas durante el tiempo que marca la dificultad.
- **Capturas:** si un fantasma toca a un Pac-Man, gana 300 puntos y el Pac-Man pierde una vida. Si el fantasma está asustado, el Pac-Man se lo come, gana 200 puntos y el fantasma pierde una vida.
- **Reaparición:** quien pierde una vida reaparece a los 2 segundos en el punto de reaparición libre más alejado de sus rivales y queda invencible 3 segundos.
- **Explosión entre fantasmas:** si dos fantasmas chocan quedan aturdidos 2 segundos y vuelven a puntos de reaparición distintos, sin perder vidas.
- **Choque entre Pac-Man:** según la configuración, rebotan (regresan a su casilla y dan la vuelta) o se atraviesan.
- **Cajas de objetos:** aparecen cada cierto tiempo en puntos fijos del mapa. Quien pasa por una con el espacio libre recibe un objeto al azar de su bando y lo usa con el botón de acción.
- **Fin del nivel:** ganan los Pac-Man si se comen todas las bolas o si eliminan a todos los fantasmas; ganan los fantasmas si eliminan a todos los Pac-Man. Cada integrante del bando ganador recibe 500 puntos extra.

1.5 Objetos

Objeto	Bando	Clave	Duración	Efecto
Turbo	Pac-Man	VELOCIDAD	5 s	Quien lo usa va 1.5 veces más rápido
Escudo	Pac-Man	ESCUDO	4 s	No puede ser atrapado ni caer en trampas
Hielo	Pac-Man	CONGELAR	3 s	Todos los fantasmas quedan inmóviles
Puntos dobles	Pac-Man	PUNTOS_DOBLES	8 s	Gana el doble por bolas y fantasmas
Ralentizar	Fantasma	RALENTIZAR	4 s	Los Pac-Man van a la mitad de velocidad
Teletransporte	Fantasma	TELETRANSPORTE	Instantáneo	Salta hasta 6 casillas hacia adelante sin atravesar muros
Trampa de hielo	Fantasma	TRAMPA	15 s	Deja una trampa; el Pac-Man que la pisa queda congelado 3 s

1.6 Configuración de la sala

Quien crea la sala es el **anfitrión**. Antes de iniciar puede ajustar las siguientes opciones, que se aplican a toda la serie de niveles:

Opción	Valores	Efecto
Dificultad	Fácil, Normal, Difícil	Velocidad de los fantasmas de la IA (0.85×, 1.00×, 1.20×) y duración del susto (10, 7 y 4 s)
Vidas	1 a 5 (por defecto 3)	Vidas de cada personaje; se reinician en cada nivel
Choque Pac-Man	Activado / Desactivado	Si los Pac-Man rebotan al chocar entre sí
Cajas	Ninguna, Pocas, Normal, Muchas	Una caja nueva cada 20, 12 o 6 segundos (o ninguna)
Bandos fijos	Activado / Desactivado	Solo multijugador: los roles se sortean en el nivel 1 y cada jugador conserva su personaje

Modo	Normal / Clásico	Clásico: los fantasmas tienen vidas infinitas y los Pac-Man solo ganan comiéndose todas las bolas
Reaparición	30, 20, 10 o 5 s	Solo en modo clásico: tiempo que tarda un fantasma en volver tras ser comido
Tu rol	Al azar, Pac-Man, Fantasma	Solo contra la IA: el bando que prefiere el jugador

Además, el anfitrión puede **bloquear** la sala para que nadie más entre y **expulsar** a un jugador desde el lobby. Un jugador expulsado no puede volver a entrar a esa sala.

1.7 Funciones de la aplicación

- Registro con correo electrónico, verificación con un código de 6 dígitos y elección del nombre de usuario (con sugerencias).
- Inicio de sesión con correo o nombre de usuario; la sesión queda guardada en el dispositivo durante 30 días.
- Recuperar la contraseña olvidada y cambiarla con un código enviado por correo.
- Lista de salas abiertas, creación de salas y unión por código.
- Partidas en tiempo real con controles táctiles (deslizar o cruceta) o teclado.
- Ranking por rol (Pac-Man, Fantasma) y ranking total; historial de partidas con el detalle por nivel.
- Opciones: volumen, cambiar nombre, cambiar contraseña y cerrar sesión.
- Configuración de la dirección del servidor desde la app (IP de la PC o túnel de Visual Studio).

2. Arquitectura

El backend sigue una **arquitectura por capas**: cada proyecto de la solución tiene una responsabilidad y solo depende de las capas inferiores. El cliente es independiente y se comunica con el backend por HTTP (REST) y WebSockets (SignalR).

2.1 Capas de la solución

Pacman (cliente)	Phaser 4 + TypeScript + Vite. Escenas, sprites, sonidos, interpolación y servicios de comunicación. Se empaqueta para Android con Capacitor.
PacmanApi	ASP.NET Core: controladores REST, hub de SignalR (`JuegoHub`), autenticación por token, servicio de correo SMTP, CORS, Swagger y registro de dependencias (`Program.cs`).
PacmanLogicaNegocios	Reglas de negocio (clases `*LN`), motor del juego (`Juego/`), IA, efectos de objetos y gestor de salas.
PacmanAccesoDatos	Entity Framework Core: `PacManDbContext`, repositorio genérico (`RepositorioAD`) y unidad de trabajo (`UnidadTrabajoEF`).
PacmanDominio	Entidades de la base, entidades tipadas (`T*`), DTO, perfil de AutoMapper, interfaces de acceso a datos y de lógica, y el modelo del juego (sala, actores, laberinto, configuración).
PacmanUtilitarios	Clase genérica `Respuesta<T>` que usan todas las capas para devolver resultados y errores.

Proyecto	Depende de
PacmanApi	PacmanLogicaNegocios, PacmanAccesoDatos, PacmanDominio, PacmanUtilitarios
PacmanLogicaNegocios	PacmanDominio, PacmanUtilitarios
PacmanAccesoDatos	PacmanDominio, PacmanUtilitarios
PacmanDominio	PacmanUtilitarios
PacmanUtilitarios	(ninguno)

2.2 Patrones de diseño

- **Repositorio genérico:** `IRepositorioAD<TEntity>` define `InsertarAsync`, `ModificarAsync`, `EliminarAsync`, `ListarAsync`, `BuscarAsync`, `ObtenerEntidadAsync` y `ContarAsync`. `RepositorioAD<TEntity>` lo implementa sobre un `DbSet`.
- **Unidad de trabajo:** `IUnidadTrabajoEF` agrupa un repositorio por tabla y vista (`TUsuario`, `TPartida`, etc.) sobre el mismo `DbContext`, de modo que una operación de negocio usa una sola conexión.
- **Respuesta uniforme:** toda operación devuelve `Respuesta<T>` con `Success`, `Data`, `Error` y, opcionalmente, `Codigo` (por ejemplo `CORREO_NO_VERIFICADO`). La API REST y el hub responden con la misma forma en JSON (`success`, `data`, `error`).
- **Entidades tipadas y AutoMapper:** las entidades de EF (`Usuario`, `Partida`…) no salen de la capa de datos; se convierten a tipos `TUsuario`, `TPartida`, etc. con el perfil `AutoMapperProfile`.

- **Inyección de dependencias:** `Program.cs` registra los servicios `*LN` como `scoped`; el gestor de salas, el notificador y el `TimeProvider` como `singleton`.
- **Clase base de lógica:** `BaseLN<T>` concentra la unidad de trabajo, el mapeador y el registro de errores (`ILogger`) que usan todas las clases de negocio.
- **Sistemas del motor:** el motor del juego se divide en sistemas estáticos (movimiento, colisiones, puntaje, objetos, reaparición) que actúan sobre un `ContextoMotor`, y en estrategias intercambiables para la IA (`IEstrategiaPersecucion`) y los objetos (`IEfectoObjeto`).
- **Servidor autoritativo:** el estado del juego solo existe en el servidor. Los clientes envían intenciones y reciben instantáneas.

2.3 Flujo de una petición REST

1. El cliente llama, por ejemplo, a `POST api/Usuario/IniciarSesion` con `ClienteApi.ts` (función `fetch`). Si hay sesión, agrega `Authorization: Bearer <token>`.
2. El controlador (`UsuarioController`) recibe el cuerpo JSON y llama al servicio de negocio inyectado (`IUsuarioLN`).
3. `UsuarioLN` valida los datos y usa `IUnidadTrabajoEF` para consultar o guardar.
4. `RepositorioAD` traduce la operación a consultas de Entity Framework Core sobre SQL Server.
5. El resultado vuelve como `Respuesta<T>`; el controlador responde `200 OK` o `400 BadRequest` con el mismo formato.

2.4 Flujo de una jugada en tiempo real

1. El cliente abre una conexión de SignalR con `ConexionTiempoReal.ts` hacia `/hubs/juego`, enviando el token.
2. Al deslizar el dedo, el cliente invoca `EnviarDireccion(codigo, direccion)` en `JuegoHub`.
3. El hub obtiene el usuario del token y entrega la intención a `GestorSalas`, que la pasa al `MotorPartida` de esa sala.
4. Cada 50 ms el ciclo de la sala avanza el motor: mueve a todos, resuelve colisiones, puntos, objetos y reapariciones.
5. El `NotificadorJuego` envía `EstadoActualizado` con un `EstadoPartidaDto` a todos los miembros de la sala (grupo de SignalR).
6. El cliente guarda los estados en un búfer y dibuja interpolando entre dos de ellos con un pequeño retraso (100 a 300 ms) para que el movimiento sea continuo.

3. Tecnologías, paquetes y herramientas

3.1 Backend (.NET)

Tecnología / paquete	Versión	Proyecto	Uso
.NET / C#	10.0 (net10.0)	Todos	Plataforma y lenguaje del backend
ASP.NET Core	10.0	PacmanApi	API REST, middleware, CORS, autenticación
ASP.NET Core SignalR	10.0 (incluido)	PacmanApi	Comunicación en tiempo real (WebSockets) del juego
Microsoft.EntityFrameworkCore	10.0.12	PacmanAccesoDatos	ORM para acceder a la base de datos
Microsoft.EntityFrameworkCore.SqlServer	10.0.12	PacmanAccesoDatos	Proveedor de EF Core para SQL Server
Microsoft.EntityFrameworkCore.Design / Tools	10.0.12	AccesoDatos, Api	Herramientas de EF Core (scaffolding)
AutoMapper	16.2.0	PacmanDominio	Conversión entre entidades y entidades tipadas/DTO
BCrypt.Net-Next	4.2.0	PacmanLogicaNegocios	Hash de contraseñas y de códigos de verificación
Microsoft.Extensions.Logging.Abstractions	10.0.12	PacmanLogicaNegocios	Registro de errores (`ILogger`)
MailKit	4.18.1	PacmanApi	Envío de correos por SMTP (Gmail)
Swashbuckle.AspNetCore	10.2.3	PacmanApi	Interfaz Swagger para probar la API
Microsoft.AspNetCore.OpenApi	10.0.9	PacmanApi	Documento OpenAPI de la API
QRCoder	1.8.1	PacmanApi	Genera el QR del túnel en la página `/conectar`

3.2 Cliente (npm)

Paquete	Versión	Tipo	Uso
phaser	4.0.0	Dependencia	Motor de juegos 2D: escenas, sprites, animaciones, entrada, sonido y renderizado WebGL/Canvas
typescript	~5.7.2	Desarrollo	Lenguaje del cliente con tipado estático
vite	^6.3.1	Desarrollo	Servidor de desarrollo con recarga en caliente y empaquetado de producción
terser	^5.39.0	Dependencia	Minificación del código en la compilación de producción
@microsoft/signalr	^10.0.11	Dependencia	Cliente de SignalR para el hub del juego
@fontsource/press-start-2p	5.2.7	Dependencia	Fuente pixelada "Press Start 2P" empaquetada (funciona sin internet)
@capacitor/core	^8.5.2	Dependencia	Núcleo de Capacitor (puente web ↔ nativo)
@capacitor/android	^8.5.2	Dependencia	Plataforma Android de Capacitor
@capacitor/cli	^8.5.2	Desarrollo	Comandos `cap sync` y `cap open`
@capacitor-mlkit/barcode-scanning	8.2.1	Dependencia	Lector de códigos QR con la cámara (Google ML Kit)

3.3 Android

Elemento	Valor
Identificador de la app	`com.pacman.multijugador`
Nombre visible	Pac-Man Multijugador
Versión	1.0.6 (versionCode 7)
SDK mínimo / objetivo / compilación	24 (Android 7.0) / 36 / 36
Gradle	8.14.3
JDK	21 a 24
Orientación	Horizontal (`sensorLandscape`)
Permisos	`INTERNET` y `CAMERA` (para escanear el QR del túnel)

3.4 Base de datos

- **Microsoft SQL Server** (instancia local o SQL Server Express) con la base `PacmanDB`.
- **SQL Server Management Studio (SSMS)** para ejecutar los scripts de la carpeta `Base De Datos`.
- Autenticación de Windows (`Trusted_Connection=True`).

3.5 Herramientas de desarrollo

Herramienta	Uso
Visual Studio 2026	Edición y depuración de la solución `Pacman.slnx`; perfiles de ejecución de la API; Dev Tunnels
Dev Tunnels (Visual Studio)	Dirección HTTPS pública para conectar celulares desde cualquier red
Visual Studio Code	Edición del cliente TypeScript (opcional)
Node.js 22 o superior	Ejecutar Vite, TypeScript y la CLI de Capacitor
Android Studio	Compilar la APK (variante debug) con el SDK de Android
Git y GitHub	Control de versiones y trabajo en equipo con ramas y pull requests (repositorio Darko0424/Pacman-proyecto)
Swagger UI	Probar los endpoints REST desde el navegador (`/swagger`)
PacmanApi.http	Peticiones de ejemplo para probar la API desde Visual Studio

4. Estructura del repositorio

```
Pacman-proyecto/
├── Pacman.slnx                  Solución de Visual Studio
├── Base De Datos/               Scripts SQL 00 a 09
├── PacmanUtilitarios/           Respuesta<T>
├── PacmanDominio/
│   ├── Clases/                  Modelos de entrada/salida (cls*)
│   ├── DTO/                     AutoMapperProfile y DTO de tiempo real
│   ├── Entidades/               Entidades de EF (y Vistas/)
│   ├── EntidadesTipadas/        T* (lo que circula entre capas)
│   ├── InterfazAD/              IRepositorioAD, IUnidadTrabajoEF
│   ├── InterfazLN/              Interfaces de negocio, IGestorSalas, INotificadorJuego
│   └── Juego/                   Modelo del juego: Actores, Mapa, Items, Salas, Interfaces
├── PacmanAccesoDatos/
│   ├── Contexto/                PacManDbContext
│   └── Implementaciones/        RepositorioAD, UnidadTrabajoEF
├── PacmanLogicaNegocios/
│   ├── Implementacion/          BaseLN y servicios *LN
│   └── Juego/                   MotorPartida, ReglasJuego, Rutas
│       ├── Controladores/       ControladorHumano, ControladorIA
│       ├── IA/                  EstrategiaBlinky, EstrategiaPinky, EstrategiaInky
│       ├── Efectos/             Un efecto por objeto
│       ├── Sistemas/            Movimiento, Colisiones, Puntaje, Objetos...
│       └── Salas/               GestorSalas, ContextoSala, AsignadorPersonajes
├── PacmanApi/
│   ├── Controllers/             8 controladores REST
│   ├── Hubs/                    JuegoHub, NotificadorJuego
│   ├── Seguridad/               AutenticacionSesion (esquema para el hub)
│   ├── Extensiones/             Lectura del token Bearer
│   ├── Servicios/               Correo SMTP, página /conectar, parada del gestor
│   └── Properties/              launchSettings.json (perfiles)
└── Pacman/                      Cliente
    ├── index.html, vite/        Página y configuración de Vite (dev y prod)
    ├── public/                  style.css, favicon y assets (sprites, sonidos)
    ├── src/game/main.ts         Configuración de Phaser y lista de escenas
    ├── src/presentacion/        escenas, sprites, ui, sonidos, animaciones
    ├── src/logica/              EstadoCliente, Interpolacion, EfectosVisuales
    ├── src/servicios/           ClienteApi, ConexionTiempoReal, Configuracion...
    ├── src/modelos/             Modelos de la API y de tiempo real
    ├── android/                 Proyecto nativo de Capacitor
    └── capacitor.config.json    Identificador y nombre de la app
```

4.1 Convenciones de código

- Todo el código, los comentarios y los nombres están en **español** (por ejemplo `IniciarSesionAsync`, `VistaLaberinto`).
- Clases de negocio terminan en `LN`; las de acceso a datos en `AD`; las entidades tipadas empiezan con `T`; las clases de entrada/salida con `cls`.
- Los métodos asíncronos terminan en `Async` y devuelven `Task<Respuesta<T>>`.
- En la base de datos las tablas y columnas usan `snake_case` (`participantes_ronda`, `id_usuario`); EF las asocia a propiedades `PascalCase`.
- El JSON viaja en `camelCase` y los enumerados como número.
- Las constantes del juego se concentran en `ReglasJuego.cs` y `ConfiguracionPartida.cs` para ajustarlas sin tocar la lógica.

5. Base de datos

La base se llama **PacmanDB** y se crea con los scripts de la carpeta `Base De Datos`. Los scripts se pueden volver a ejecutar sin duplicar ni borrar datos: crean solo lo que falta.

5.1 Scripts y orden de ejecución

Script	Contenido
00_BorrarBaseDatos.sql	Solo desarrollo: borra la base completa cuando cambia la estructura. Después se ejecutan del 01 en adelante.
01_CrearBaseDatos.sql	Crea la base PacmanDB si no existe.
02_Tablas.sql	Crea las 9 tablas con sus llaves primarias, valores por defecto y restricciones CHECK.
03_Relaciones.sql	Agrega las llaves foráneas y sus reglas de borrado en cascada.
04_Indices.sql	Crea los índices de búsqueda y los índices únicos.
05_Vistas.sql	Crea las vistas de historial, progreso y ranking.
06_DatosCatalogos.sql	Inserta personajes, dificultades, objetos y los 5 niveles (laberintos en JSON).
07_DatosPrueba.sql	Reservado. Los usuarios se crean desde la API porque la contraseña se guarda con BCrypt.
08_VerificacionCorreo.sql	Agrega a `usuarios` las columnas de verificación de correo y del código de 6 dígitos.
09_PropositoCodigo.sql	Agrega `proposito_codigo` (VERIFICAR, RECUPERAR o CAMBIAR) para que un código sirva solo para lo que se pidió.

5.2 Tablas

usuarios

Columna	Tipo	Descripción
id_usuario	INT IDENTITY, PK	Identificador
nombre_usuario	VARCHAR(50), único	Nombre visible (temporal hasta que el jugador lo elige)
correo_electronico	VARCHAR(100), único	Correo en minúsculas
contrasena_hash	VARCHAR(255)	Hash BCrypt de la contraseña
fecha_registro	DATETIME2	Por defecto GETDATE( )
correo_verificado	BIT	1 cuando confirmó el código
nombre_definido	BIT	0 mientras tiene el nombre temporal
codigo_verificacion_hash	VARCHAR(255) NULL	Hash BCrypt del código de 6 dígitos vigente
codigo_expira	DATETIME2 NULL	Vencimiento del código (15 minutos)
codigo_enviado	DATETIME2 NULL	Último envío (limita reenvíos a uno por minuto)
intentos_codigo	TINYINT	Intentos fallidos (máximo 5)
proposito_codigo	VARCHAR(10) NULL	VERIFICAR, RECUPERAR o CAMBIAR

sesiones_moviles

Columna	Tipo	Descripción
id_sesion	INT IDENTITY, PK	Identificador
id_usuario	INT, FK → usuarios	Dueño de la sesión (borrado en cascada)
token_dispositivo	VARCHAR(255)	Hash SHA-256 del token (nunca el token)
ultimo_acceso	DATETIME2	Última vez que se usó; la sesión vence a los 30 días

Catálogos: personajes, dificultades, niveles y objetos

Tabla	Columnas principales	Contenido
personajes	id_personaje (TINYINT), nombre, rol (PACMAN/FANTASMA), color	5 personajes (ver 1.2)
dificultades	id_dificultad, nombre, multiplicador_velocidad_ia, duracion_asustado_seg	FACIL 0.85 / 10 s, NORMAL 1.00 / 7 s, DIFICIL 1.20 / 4 s
niveles	id_nivel, numero (único), nombre, laberinto_json (ISJSON), velocidad_base	5 laberintos de 31 × 28 casillas
objetos	id_objeto, nombre, bando, efecto, duracion_seg	7 objetos (ver 1.5)

Partidas, rondas y participantes

Tabla	Columnas principales	Reglas
partidas	id_partida, codigo_sala, modo (MULTIJUGADOR/CONTRA_IA), id_dificultad, estado, fecha_inicio, fecha_fin	estado: EN_CURSO, FINALIZADA o ABANDONADA; fecha_fin obligatoria si no está en curso
rondas	id_ronda, id_partida, id_nivel, bando_ganador, fecha_inicio, fecha_fin	Una ronda por nivel de cada partida (único partida + nivel); bando_ganador NULL si se interrumpió
participantes_ronda	id_participante, id_ronda, id_usuario, id_personaje, es_ia, puntuacion, fantasmas_comidos, pacmans_atrapados, objetos_usados, abandono	Un personaje por ronda; si es IA no tiene usuario; solo un humano puede figurar como abandono

5.3 Relaciones

Llave foránea	Desde	Hacia	Al borrar
FK_sesiones_usuarios	sesiones_moviles.id_usuario	usuarios	Cascada
FK_partidas_dificultades	partidas.id_dificultad	dificultades	Restringido
FK_rondas_partidas	rondas.id_partida	partidas	Cascada
FK_rondas_niveles	rondas.id_nivel	niveles	Restringido
FK_participantes_rondas	participantes_ronda.id_ronda	rondas	Cascada
FK_participantes_usuarios	participantes_ronda.id_usuario	usuarios	Cascada
FK_participantes_personajes	participantes_ronda.id_personaje	personajes	Restringido

En resumen: un **usuario** tiene muchas sesiones y muchas participaciones; una **partida** tiene una dificultad y hasta 5 **rondas** (una por nivel); cada ronda tiene hasta 5 **participantes**, uno por personaje.

5.4 Índices

Índice	Tabla y columnas	Para qué
IX_sesiones_token	sesiones_moviles (token_dispositivo)	Validar el token en cada petición
IX_partidas_fecha	partidas (fecha_inicio DESC)	Historial ordenado por fecha
UX_partidas_sala_inicio	partidas (codigo_sala, fecha_inicio), único	Evitar guardar dos veces la misma serie
UX_participantes_usuario	participantes_ronda (id_ronda, id_usuario), único	Un usuario aparece una vez por ronda
IX_participantes_usuario	participantes_ronda (id_usuario)	Historial y ranking por usuario

5.5 Vistas

El progreso y los rankings se calculan siempre desde las rondas, así nunca quedan desactualizados. Las partidas contra la IA cuentan para el historial y el progreso, pero no para los rankings.

Vista	Contenido
vw_historial_usuario	Una fila por usuario y partida: suma de puntos y estadísticas de todos sus niveles
vw_historial_rondas	Detalle del historial: una fila por usuario y nivel jugado
vw_progreso_jugador	Progreso general de cada usuario (solo partidas finalizadas, incluye contra la IA)
vw_ranking	Ranking por rol: mejor puntuación en un nivel con ese rol (solo multijugador)
vw_ranking_total	Ranking total: suma de puntos de ambos roles (solo multijugador)

5.6 Niveles y formato del laberinto

N.º	Nombre	Diseño	Bolas	De poder
1	Laberinto clásico	Sectores simétricos y circuito central con dos accesos	281	4
2	Doble anillo	Dos anillos alrededor de un circuito interior	275	4
3	Cuatro patios	Cuatro sectores unidos por pasos centrales y laterales	313	4
4	Pasajes alternos	Circuitos desplazados con cruces a diferentes alturas	217	4
5	Encrucijada	Dos ejes verticales y pasos horizontales	287	4

Cada laberinto se guarda en `niveles.laberinto_json` con el **contrato JSON versión 2**. Todos miden 31 filas por 28 columnas y tienen `velocidad_base = 1.00`.

Propiedad	Contenido
version	Siempre 2; otras versiones se rechazan
filas, columnas	Deben coincidir con la matriz
casillas	Una cadena por fila: # muro, `.` bola, o bola de poder, espacio = suelo sin bola
puntosReaparicion	Seis casillas `{fila, columna}` donde se reaparece
puntosCajas	Casillas donde aparecen las cajas de objetos (4 por mapa)
posicionesIniciales	Cinco objetos `{idPersonaje, fila, columna}`, uno por personaje

```
{
  "version": 2,
  "filas": 31,
  "columnas": 28,
  "casillas": [
    "############################",
    "#..........................#",
    "#o........................o#",
    ...
  ],
  "puntosReaparicion": [ {"fila": 1, "columna": 6}, ... ],
  "puntosCajas":       [ {"fila": 5, "columna": 6}, ... ],
  "posicionesIniciales": [ {"idPersonaje": 1, "fila": 25, "columna": 6}, ... ]
}
```

`SistemaNiveles` convierte el JSON en un `Laberinto` con casillas y bolas propias para cada sala. Rechaza con `FormatException` la geometría desconectada, las coordenadas inválidas, la falta de bolas y las propiedades desconocidas.

6. Backend: PacmanApi

`PacmanApi` es el proyecto que se ejecuta. Su archivo `Program.cs` arma la aplicación: registra los servicios, configura CORS, autenticación, Swagger y SignalR, y publica los controladores y el hub.

6.1 Registro de servicios (Program.cs)

Servicio	Implementación	Ciclo de vida
PacManDbContext	EF Core con `UseSqlServer` y la cadena `ConnectionStrings:PacmanDb`	Scoped
IUnidadTrabajoEF	UnidadTrabajoEF	Scoped
IDificultadLN, IObjetoLN, INivelLN, IPersonajeLN	Catálogos	Scoped
IUsuarioLN	Registro, sesiones, verificación y contraseñas	Scoped
IHistorialLN, IRankingLN, IPartidaLN	Historial, ranking y persistencia de partidas	Scoped
IServicioCorreo	ServicioCorreoSmtp (MailKit), opciones en la sección `Correo`	Singleton
INotificadorJuego	NotificadorJuego (sobre `IHubContext<JuegoHub>`)	Singleton
GestorSalas / IGestorSalas	Salas en memoria y ciclo de cada partida	Singleton
TimeProvider	TimeProvider.System (reemplazable en pruebas)	Singleton
ParadaGestorSalas	IHostedService que cierra el gestor al apagar el servidor	Hosted service
AutoMapper	Perfil `AutoMapperProfile` del dominio	—

> El gestor de salas y el notificador deben ser **singleton**: si fueran scoped, cada solicitud vería salas distintas. `ParadaGestorSalas` los cierra antes de que se destruya el contenedor de dependencias, para poder guardar como abandonadas las partidas que estaban en curso.

6.2 Endpoints REST

Todas las rutas empiezan con `api/{Controlador}`. Responden `Respuesta<T>` en JSON. Las marcadas con **Token** requieren el encabezado `Authorization: Bearer <token>` obtenido al iniciar sesión.

Usuario (api/Usuario)

Método y ruta	Token	Descripción
POST Registrar	No	Crea la cuenta con correo y contraseña, le asigna un nombre temporal y envía el código de verificación
POST VerificarCorreo	No	Confirma el código de 6 dígitos; devuelve la sesión iniciada
POST ReenviarCodigo	No	Envía otro código (máximo uno por minuto)
POST IniciarSesion	No	Con correo o nombre de usuario y contraseña. Si el correo no está verificado responde con el código `CORREO_NO_VERIFICADO`
GET Sesion	Sí	Valida el token guardado y devuelve el usuario
GET SugerirNombres?nombre=	No	Propone 5 nombres disponibles parecidos al pedido
POST AsignarNombre	Sí	Define o cambia el nombre de usuario
POST SolicitarRecuperacion	No	Envía un código para recuperar la contraseña olvidada
POST RestablecerContrasena	No	Con el código de recuperación, define la nueva contraseña y cierra las demás sesiones
POST SolicitarCambioContrasena	Sí	Envía un código para cambiar la contraseña desde Opciones
POST CambiarContrasena	Sí	Con el código, cambia la contraseña y deja solo la sesión de este dispositivo
POST CerrarSesion	Sí	Elimina la sesión del dispositivo

Catálogos

Controlador	Rutas	Descripción
Dificultad	GET Listar · GET Obtener/{id} · GET Buscar?texto=	Niveles de dificultad
Nivel	GET Listar · GET Obtener/{id} · GET Buscar?texto=	Laberintos (incluye el JSON)
Objeto	GET Listar · GET Obtener/{id} · GET Buscar?texto= · GET Bando/{bando}	Objetos y su efecto
Personaje	GET Listar · GET Obtener/{id} · GET Buscar?texto=	Los 5 personajes

Historial, ranking y partidas

Método y ruta	Token	Descripción
GET Historial/Listar	Sí	Partidas del usuario con su total
GET Historial/Rondas/{idPartida}	Sí	Detalle por nivel de una partida
GET Historial/Progreso	Sí	Estadísticas generales del usuario
GET Ranking/Listar/{rol}	No	Ranking de PACMAN o FANTASMA
GET Ranking/Total	No	Ranking total (ambos roles)
GET Ranking/MiPosicion/{rol}	Sí	Posición del usuario en el ranking de un rol
GET Ranking/MiPosicionTotal	Sí	Posición del usuario en el ranking total
GET Partida/Obtener/{id}	No	Datos de una partida
GET Partida/Rondas/{id}	No	Rondas de una partida
GET Partida/Participantes/{id}	No	Participantes de una ronda

En total son **36 endpoints** en 8 controladores. Se pueden probar en Swagger (`http://localhost:5253/swagger`) o con el archivo `PacmanApi.http`.

```
POST http://localhost:5253/api/Usuario/IniciarSesion
Content-Type: application/json

{ "usuario": "jugador@gmail.com", "contrasena": "Pacman#2026" }

200 OK
{ "success": true, "data": { "token": "…", "usuario": { … } }, "error": "" }
```

6.3 Hub de tiempo real (SignalR)

El hub `JuegoHub` está en `/hubs/juego` y exige el token de sesión (esquema `AutenticacionSesion`). Solo recibe intenciones; nunca posiciones ni estados. Cada método responde con `Respuesta<T>`.

Métodos (cliente → servidor)

Método	Parámetros	Descripción
ListarSalas	—	Salas disponibles (máximo 5 a la vez en el servidor)
CrearSala	—	Crea una sala con código de 6 caracteres; quien la crea es el anfitrión
UnirseSala	codigo	Entra al lobby (máximo 4 jugadores, sala no bloqueada)
SalirSala	codigo	Libera la plaza; en pleno nivel la IA termina por él
ReconectarSala	codigo	Recupera la plaza con una conexión nueva y recibe sala, mapa y estado

ReanudarSala	—	Igual que la anterior cuando el cliente ya no recuerda el código
ConfigurarSala	codigo, ConfiguracionSalaDto	Solo el anfitrión, en el lobby
ElegirRol	codigo, "PACMAN" / "FANTASMA" / null	Preferencia para jugar contra la IA
BloquearSala	codigo, bloqueada	Solo el anfitrión: impide que entren más jugadores
ExpulsarJugador	codigo, idJugador	Solo el anfitrión, en el lobby
IniciarPartida	codigo	Solo el anfitrión: sortea roles y empieza el nivel 1
EnviarDireccion	codigo, direccion (1 arriba, 2 abajo, 3 izquierda, 4 derecha)	Se guarda hasta que el personaje pueda girar
UsarObjeto	codigo	Botón de acción
ObtenerSala, ObtenerNivel, ObtenerEstado	codigo	Instantáneas para recuperarse si se perdió un aviso

Avisos (servidor → cliente)

Evento	Datos	Cuándo
SalaActualizada	SalaDto	Cambios de jugadores, configuración o estado (1 lobby, 2 jugando, 3 transición, 4 finalizada)
NivelIniciado	NivelDto	Al empezar cada nivel: mapa, bolas, puntos de cajas y configuración
EstadoActualizado	EstadoPartidaDto	Unas 20 veces por segundo: personajes (posición, dirección y avance), bolas comidas, cajas, trampas, efectos y eventos
SesionReemplazada	—	La cuenta se abrió en otro dispositivo o se cerraron las sesiones
Expulsado	codigoSala	El anfitrión expulsó a este jugador

Los avisos de una sala se envían al grupo de SignalR `sala-{CODIGO}`. Los JSON viajan en camelCase y los enumerados como número. Los avisos no garantizan entrega exactamente una vez: si el cliente se reconecta llama a `ReconectarSala` o `ReanudarSala` y continúa con las instantáneas que recibe.

6.4 Autenticación

- Al iniciar sesión se genera un token aleatorio de 32 bytes. Al cliente se le entrega el token; en `sesiones_moviles` se guarda solo su **hash SHA-256**.
- La sesión vale 30 días desde el último acceso.
- Los controladores leen el token del encabezado `Authorization: Bearer` (`SolicitudExtensiones.ObtenerToken`) y lo validan con `IUsuarioLN.ValidarSesionAsync`.
- El hub usa el esquema `AutenticacionSesion`. Como en WebSockets el navegador no permite encabezados, también acepta `?access_token=`, pero solo en rutas `/hubs`.
- Sin token o con un token vencido, el hub responde 401.

6.5 Correo electrónico

`ServicioCorreoSmtp` envía los códigos con **MailKit** a través de Gmail (`smtp.gmail.com`, puerto 587, STARTTLS). La sección `Correo` de la configuración tiene `Servidor`, `Puerto`, `Usuario`, `Contrasena` y `NombreRemitente`. El usuario y la contraseña de aplicación de Gmail se guardan como **secretos de usuario** (no en Git).

> Para pruebas sin la clave de Gmail: en el entorno Development, si falta el usuario o la contraseña del correo, el código se escribe como advertencia en la consola de la API en lugar de enviarse.

6.6 Página /conectar

Con el perfil `red-local`, la API abre en la PC la página `http://localhost:5253/conectar` (`PaginaConexionLocal.cs`). Muestra:

- **Internet: túnel de Visual Studio:** si hay un Dev Tunnel activo (variable `VS_TUNNEL_URL`), su dirección HTTPS y un **código QR** para escanearlo desde la app.
- **Red local:** la IP de cada adaptador de red de la PC (por ejemplo `http://192.168.1.4:5253`) para escribirla a mano en la app cuando todos están en la misma Wi-Fi.

El QR contiene un enlace con esquema propio (`pacman://servidor?url=…`) para que la app no lo confunda con otros códigos. La página no comparte usuarios, contraseñas ni datos de la base.

6.7 CORS

El cliente de SignalR envía credenciales, por eso los orígenes deben ser explícitos. En desarrollo se acepta cualquier origen. Fuera de desarrollo se usan los de `Cors:Origenes`: `http://localhost:8080` (Vite) y `https://localhost`, `http://localhost`, `capacitor://localhost` (Capacitor).

7. Motor del juego

El motor está en `PacmanLogicaNegocios/Juego`. Cada nivel de cada sala tiene su propio `MotorPartida` con su laberinto, actores, dificultad y objetos. Simula en pasos fijos (ticks) de **50 ms** (20 por segundo) y no es seguro para varios hilos: el gestor de salas serializa todas sus llamadas.

7.1 Orden de un tick

1. Vencer los efectos de objetos que terminaron (`SistemaObjetos.VencerEfectos`).
2. Reaparecer a quienes cumplieron su demora (`SistemaReaparicion`).
3. Leer las intenciones: humanos (`ControladorHumano`) e IA (`ControladorIA`); usar objetos solicitados.
4. Mover a todos los actores (`SistemaMovimiento`).
5. Resolver choques entre actores (`SistemaColisiones`).
6. Activar trampas de hielo pisadas, comer bolas, recoger cajas y hacer aparecer cajas nuevas.
7. Si un bando ganó, cerrar el nivel y otorgar la bonificación (`SistemaPuntaje.CerrarNivel`).

7.2 Sistemas

Sistema	Responsabilidad
SistemaMovimiento	Movimiento por casillas: cada actor acumula avance según su velocidad y pasa a la siguiente casilla al completar 1. Solo gira al llegar a una casilla; la última dirección pedida se guarda
SistemaColisiones	Dos actores chocan si terminan en la misma casilla o si se cruzaron en el tick. Resuelve capturas, fantasmas comidos, explosiones y rebotes
SistemaPuntaje	Bolas comidas, bolas de poder (asustan según la dificultad) y bonificación del bando ganador
SistemaObjetos	Aparición y recogida de cajas, uso de objetos, trampas de hielo y vencimiento de efectos
SistemaReaparicion	Reaparece tras la demora en el punto libre más alejado de los rivales (empates al azar)
SistemaNiveles	Convierte el JSON del catálogo en un `Laberinto` nuevo para cada sala
ControladorModos	Ciclo de los fantasmas de la IA: dispersión (cada uno a su esquina) y persecución
Rutas	Vecinos, direcciones y distancias reales por búsqueda en anchura (BFS), rodeando muros

7.3 Velocidades y tiempos

Valor	Cantidad
Duración del tick	50 ms
Velocidad de Pac-Man	6 casillas/s (87 % mientras come)
Velocidad de los fantasmas	5.6 casillas/s × multiplicador de dificultad (solo los de la IA)
Fantasma asustado	50 % de su velocidad
Velocidad máxima	19 casillas/s (nadie avanza más de una casilla por tick)
Reaparición	2 s (fantasmas en modo clásico: 30, 20, 10 o 5 s)
Invencibilidad al reaparecer	3 s
Aturdimiento por explosión	2 s
Congelado por trampa de hielo	3 s
Transición entre niveles	5 s

Ciclo de modos de la IA	Dispersión 4 s → persecución 20 s → dispersión 3 s → persecución 20 s → dispersión 3 s → persecución hasta el final

7.4 Puntaje

Acción	Puntos
Bola normal	10
Bola de poder	50
Comerse un fantasma asustado	200
Atrapar a un Pac-Man	300
Ganar el nivel (cada integrante del bando)	500
Con Puntos dobles activo	×2 en bolas y fantasmas

7.5 Inteligencia artificial

- **Blinky (rojo):** persigue directamente al Pac-Man más cercano.
- **Pinky (rosa):** apunta 4 casillas por delante del Pac-Man más cercano para emboscarlo.
- **Inky (celeste):** toma el punto 2 casillas por delante del Pac-Man y duplica el vector desde Blinky hasta ese punto. Sin Blinky en juego, el vector parte del propio Inky.
- **Dispersión:** Blinky va a la esquina superior derecha, Pinky a la superior izquierda e Inky a la inferior derecha.
- **Asustados:** se alejan de los Pac-Man.
- Los fantasmas no dan media vuelta salvo que no tengan otra salida, y evitan pisar la casilla de otro fantasma (penalización de 4) para no explotar.
- **Pac-Man de la IA:** come bolas, huye si un fantasma peligroso está a 6 casillas o menos y persigue fantasmas asustados a 12 casillas o menos.
- La IA usa su objeto cuando tiene un rival a 6 casillas o menos.
- La IA también mueve a los humanos desconectados o que abandonaron (`Entidad.ControlIa`).

7.6 Objetos y efectos

Cada objeto es una clase que implementa `IEfectoObjeto` (`MultiplicadorVelocidad`, `MultiplicadorPuntos`, `Protege`, `Aplicar`, `Retirar`): `EfectoVelocidad`, `EfectoEscudo`, `EfectoCongelar`, `EfectoPuntosDobles`, `EfectoRalentizar`, `EfectoTeletransporte` y `EfectoTrampa`.

Para agregar un objeto nuevo: crear la clase del efecto, registrarla en `MotorPartida.EfectosPredeterminados` e insertar su fila en la tabla `objetos` con la misma clave en la columna `efecto`. Los objetos cuyo efecto no tiene clase se ignoran.

7.7 Modelo del dominio del juego

Clase	Descripción
Entidad	Base de los actores: posición, dirección, avance, vidas (o vidas infinitas), invencibilidad, congelamiento, objeto guardado, control de la IA y estadísticas
PacMan, Fantasma	Actores concretos; el fantasma agrega estados asustado y aturdido
Laberinto, Casilla	Matriz de casillas transitables, bolas, puntos de reaparición y de cajas
Punto, PuntoPoder, CajaObjeto, Trampa, EfectoActivo	Elementos del mapa y efectos en curso

EstadoPartida	Estado completo de un nivel: actores, bolas, cajas, trampas, efectos, tiempo de simulación y bando ganador
ConfiguracionSala	Opciones del anfitrión (inmutable: cada cambio crea una nueva)
ConfiguracionPartida	Constantes: 5 niveles, 2 Pac-Man, 3 fantasmas, máximo 4 humanos, 6 puntos de reaparición
Sala, JugadorConectado	Lobby, jugadores, anfitrión, estado de la serie y personajes asignados
EventoJuego	Sucesos de un tick para sonidos y animaciones (bola comida, captura, explosión, reaparición, caja, objeto, fin)

8. Gestor de salas

`GestorSalas` (`PacmanLogicaNegocios/Juego/Salas`) administra las salas en memoria durante la vida del servidor. Admite hasta **5 salas** a la vez; cada usuario y cada conexión solo pueden estar en una sala. Los códigos de sala tienen 6 caracteres de un alfabeto sin letras ni números que se confundan (sin I, O, 0 ni 1).

8.1 Estados de una sala

Estado	Valor	Qué pasa
Lobby	1	Se unen jugadores; el anfitrión configura, bloquea o expulsa e inicia
Jugando	2	El motor avanza cada 50 ms y se envían los estados
Transición	3	5 s entre niveles: resultados y asignación de personajes del siguiente nivel
Finalizada	4	Terminó la serie (o todos salieron); los que siguen pueden volver al lobby

8.2 Inicio y persistencia

1. El anfitrión llama a `IniciarPartida`. Se carga el nivel 1 desde el catálogo y `AsignadorPersonajes` sortea los roles.
2. Se guarda la partida con `PartidaLN.IniciarAsync` (estado EN_CURSO) y se registra su identificador en la sala.
3. Se crea el `MotorPartida` y se envía `NivelIniciado` a todos.
4. Al terminar cada nivel se guarda la ronda y sus participantes (`PartidaLN.GuardarRondaAsync`) y luego empieza la transición.
5. Después del nivel 5 la partida se cierra como **FINALIZADA**. Si todos salen antes, se guarda el nivel en curso sin ganador y se cierra como **ABANDONADA**.

Los guardados que fallan se reintentan una vez por segundo sin cambiar fechas ni claves, de modo que no se duplican rondas.

8.3 Asignación de personajes

- **Contra la IA:** el jugador toma su rol preferido o uno al azar; si es fantasma, se elige uno de los tres al azar. El rol se conserva en todos los niveles.
- **Multijugador:** los humanos se reparten al azar entre los personajes 1 a 4. Con **bandos fijos** activado, el sorteo solo ocurre en el nivel 1 y cada jugador conserva su personaje; de lo contrario se sortea en cada nivel.
- Los personajes sin humano los controla la IA. En **modo clásico** los fantasmas se crean con vidas infinitas.

8.4 Ciclo, concurrencia y conexiones

- Cada sala tiene una tarea propia con un `PeriodicTimer` de 50 ms. El tiempo transcurrido lo da `TimeProvider`.
- Todas las operaciones de una sala pasan por la misma **compuerta** (semáforo); el candado global solo protege los índices y nunca espera operaciones externas. Una sala lenta no frena a las demás.
- Las consultas a catálogos y los guardados abren un ámbito de dependencias corto; el gestor nunca comparte un `DbContext` entre salas.

- Una **desconexión** conserva la plaza y la IA mueve el personaje; al **reconectar** se envían sala, mapa y estado a la conexión nueva. Una **salida voluntaria** libera la plaza.
- Si la misma cuenta se conecta desde otro dispositivo, la conexión anterior recibe `SesionReemplazada`.
- Al apagar el servidor, `DisposeAsync` detiene los ciclos y guarda como abandonadas las series en curso.

9. Cliente: Phaser y TypeScript

El cliente vive en la carpeta `Pacman`. `index.html` carga `src/main.ts`, que crea el juego de Phaser (`src/game/main.ts`) dentro de `#game-container`. La resolución de referencia es 1024 × 768 y se adapta a cualquier pantalla con `Scale.FIT`; los sprites son pixel art (`pixelArt: true`) y los campos de texto son elementos HTML sobre el canvas (`dom.createContainer`).

9.1 Escenas

Phaser organiza el juego en **escenas**. Cada pantalla es una clase que extiende `Scene` en `src/presentacion/escenas`:

Escena	Función
EscenaCarga	Carga sprites y sonidos, registra las animaciones, aplica el volumen guardado y pasa al inicio de sesión
EscenaLogin	Inicio de sesión (correo o usuario y contraseña); acceso a SERVIDOR, CREAR CUENTA y recuperar contraseña
EscenaServidor	Dirección de la API: escribir la IP o URL del túnel, escanear el QR del túnel, GUARDAR SERVIDOR o PREDETERMINADA
EscenaRegistro	Crear cuenta con correo y contraseña (valida las reglas antes de enviar)
EscenaVerificacion	Ingresar el código de 6 dígitos y reenviarlo
EscenaNombre	Elegir el nombre de usuario con sugerencias
EscenaContrasena	Recuperar la contraseña (sin sesión) o cambiarla (desde Opciones) con código por correo
EscenaMenu	Menú principal: JUGAR EN LINEA, RANKING, HISTORIAL, OPCIONES y CERRAR SESION
EscenaOpciones	Volumen (silencio, 25 %, 50 %, 75 %, 100 %), cambiar nombre y cambiar clave
EscenaListaSalas	Lista de salas (se actualiza periódicamente), CREAR SALA y UNIRSE por código
EscenaSala	Lobby: plazas de jugadores, configuración del anfitrión, bloquear, expulsar e INICIAR PARTIDA
EscenaJuego	La partida: laberinto, personajes, HUD, controles, sonidos y carteles entre niveles
EscenaRanking	Pestañas PAC-MAN, FANTASMA y TOTAL con paginación y la posición propia
EscenaHistorial	Partidas jugadas y su detalle por nivel
EscenaGaleriaSprites	Solo desarrollo (`?sprites`): muestra todos los sprites y animaciones

`FlujoSesion.ts` reúne lo común a las pantallas de sesión (encabezado, avisos y decisión de a qué pantalla ir después de iniciar sesión).

9.2 Servicios

Archivo	Responsabilidad
ClienteApi.ts	Llamadas REST con `fetch`; guarda la sesión (token y usuario) en `localStorage` (`pacman.sesion`); traduce fallos de red a `Respuesta` con error "No se pudo conectar con el servidor."
ConexionTiempoReal.ts	Única conexión de SignalR con `/hubs/juego` (`withAutomaticReconnect`). Las escenas se suscriben a los avisos con `on()` y cancelan al cerrarse
Configuracion.ts	Dirección de la API: la guardada en el dispositivo (`pacman.servidor`), o `VITE_API_URL` al compilar para Android, o el mismo equipo que sirve la página (puerto 5253) en el navegador. `normalizarUrlApi` acepta IP, IP:puerto o URL (los túneles `*.devtunnels.ms` se completan con https)
EscanerServidorQr.ts	Lector de QR con ML Kit; solo acepta enlaces `pacman://servidor?url=` que apunten a un túnel `https://…devtunnels.ms`

9.3 Lógica del cliente

- **EstadoCliente:** guarda lo último que mandó el servidor (sala, nivel y estados) y quién es el jugador del dispositivo. No calcula reglas.
- **Interpolacion:** el servidor manda unos 20 estados por segundo. Para que el movimiento se vea continuo se dibuja un poco en el pasado (entre 100 y 300 ms, ajustado según cómo llegan los paquetes), siempre entre dos estados recibidos. Guarda hasta 30 estados y no interpola saltos mayores a 1.5 casillas (reapariciones o teletransportes).
- **EfectosVisuales:** cómo se ve un personaje según los efectos activos (turbo, escudo, congelado, etc.) y la descripción corta de cada objeto para el HUD.

9.4 Presentación

Carpeta	Contenido
sprites	`CatalogoSprites` (texturas y cuadros), `SpritePacMan`, `SpriteFantasma` (cuerpo animado y cara que mira), `VistaActor` (sprite, nombre, parpadeos, rastro de velocidad) y `VistaLaberinto` (dibuja muros con bordes redondeados, bolas, cajas y trampas)
ui	`Boton`, `Selector`, `CampoTexto`, `Panel`, `Paleta` (colores sacados de los sprites), `HUD`, `IndicadorEfectos`, `ControlesTactiles`, `DistribucionJuego` y `PantallaAdaptable`
sonidos	`CatalogoSonidos`, `GestorSonido` (sonido para cada evento del servidor) y `Volumen` (guardado en el dispositivo)
animaciones	Animaciones globales registradas una sola vez en `EscenaCarga`

Recursos

- **Sprites** (`public/assets/sprites`): `pacman-amarillo.png`, `pacman-verde.png`, `fantasmas.png`, `items.png`, `tiles.png` y `laberinto-clasico.png`. Casillas e ítems de 16 px y actores de 32 px.
- **Sonidos** (`public/assets/sonidos`): `waka`, `comer-fantasma`, `comer-fruta`, `muerte`, `vida`, `intermision` y `nivel-completado` en MP3.
- **Fuente:** Press Start 2P, empaquetada con Fontsource.

9.5 Pantalla de juego

- **Distribución:** panel izquierdo (nivel, rol, jugadores con vidas y puntos, efectos y cruceta), mapa al centro y panel derecho (puntaje, vidas, botón del objeto, silencio y salir).
- **Laberinto:** las casillas se ensanchan hasta 1.4 veces (`ESTIRAMIENTO_MAXIMO`) en pantallas anchas para no dejar franjas vacías; los sprites usan el lado menor para no deformarse.
- **Controles:** deslizar el dedo sobre el mapa (umbral de 24 px), cruceta en pantalla, teclado (flechas o W, A, S, D) y botón de objeto.
- **HUD:** las vidas de los fantasmas en modo clásico se muestran como "INF".
- **Carteles:** presentación del nivel y rol, cuenta regresiva de la transición y resultados al final.

9.6 Pantalla adaptable

`PantallaAdaptable.ts` ajusta el lienzo a la pantalla sin barras ni franjas: la interfaz de 1024 × 768 siempre cabe completa y el espacio sobrante agranda el lienzo. También mueve el contenido cuando aparece el teclado del celular y filtra los eventos de mouse que el navegador simula después de cada toque, para que los botones y flechas no se activen dos veces.

10. Seguridad

10.1 Contraseñas

- Se guardan con **BCrypt** (`EnhancedHashPassword`, factor de trabajo 12). Nunca se guardan en texto plano.
- Reglas (iguales en el cliente y en la API): de 8 a 128 caracteres, sin espacios, al menos una mayúscula, una minúscula, un número y un símbolo, y no puede contener la parte del correo antes de la @. Se aplican al registrarse, recuperar y cambiar la contraseña.

10.2 Códigos por correo

- Código de 6 dígitos guardado como hash BCrypt (factor 10).
- Vence a los 15 minutos, se puede reenviar una vez por minuto y admite 5 intentos fallidos.
- La columna `proposito_codigo` asegura que un código de verificación no sirva para recuperar o cambiar la contraseña, y viceversa.
- Al pedir recuperación no se revela si el correo existe.

10.3 Sesiones

- Token aleatorio de 32 bytes; en la base solo se guarda su hash SHA-256.
- Vigencia de 30 días. Restablecer o cambiar la contraseña cierra las demás sesiones.
- El hub solo acepta el token en la URL en rutas `/hubs`.
- Si la cuenta se abre en otro dispositivo, el anterior recibe `SesionReemplazada` y vuelve al inicio de sesión.

10.4 Juego

- El servidor es la única autoridad: el cliente no envía posiciones ni puntos, solo intenciones.
- Las acciones de la sala validan la identidad (anfitrión, miembro de la sala) en el servidor.

10.5 Configuración y secretos

- Las credenciales del correo se guardan con **User Secrets** de .NET (fuera del repositorio).
- La APK no incluye contraseñas de SQL ni del correo.
- El permiso de tráfico HTTP sin cifrar solo está habilitado en la variante **debug** de Android, para las pruebas en red local.
- Un túnel público permite que cualquiera llegue a la API: debe usarse solo durante las pruebas y desactivarse al terminar.

11. Instalación y configuración

11.1 Requisitos

Software	Versión
Visual Studio con la carga de trabajo ASP.NET y desarrollo web	2026 (compatible con .NET 10)
.NET SDK	10.0
SQL Server (o Express) y SSMS	Cualquier versión reciente
Node.js	22 o posterior
Android Studio, Android SDK Platform 36 y JDK	JDK 21 a 24 (solo para la APK)

11.2 Base de datos

1. Abrir SQL Server Management Studio y conectarse a la instancia local.
2. Ejecutar en orden los scripts `01` a `09` de la carpeta `Base De Datos`.
3. Si la estructura cambió en una versión nueva, ejecutar antes `00_BorrarBaseDatos.sql` (borra toda la base) y luego del `01` al `09`.
4. Crear los usuarios desde la app o con `POST api/Usuario/Registrar` (no por script, por el hash de BCrypt).

11.3 Configuración de la API

La cadena de conexión está en `PacmanApi/appsettings.json` (clave `ConnectionStrings:PacmanDb`). Cada integrante ajusta el nombre de su servidor; el valor va en una sola línea:

```
Server=NOMBRE-PC\\INSTANCIA;Database=PacmanDB;Trusted_Connection=True;
Encrypt=False;TrustServerCertificate=True
```

Las credenciales del correo (cuenta de Gmail con contraseña de aplicación) se guardan como secretos de usuario, desde la carpeta `PacmanApi`:

```
dotnet user-secrets set "Correo:Usuario" "cuenta@gmail.com"
dotnet user-secrets set "Correo:Contrasena" "abcd efgh ijkl mnop"
```

Perfiles de ejecución (launchSettings.json)

Perfil	Escucha en	Uso
http	http://localhost:5253	Desarrollo en la misma PC; abre Swagger
red-local	http://0.0.0.0:5253	Celulares en la red local o por túnel; abre Swagger y la página /conectar
https	https://localhost:7149 y http://localhost:5253	Pruebas con HTTPS local

```
cd PacmanApi
dotnet run --launch-profile red-local
```

11.4 Cliente web

```
cd Pacman
npm ci
npm run dev      # http://localhost:8080 (también desde la red local)
npm run build    # compilación de producción en dist/
```

En el navegador, el cliente busca la API en el mismo equipo que sirve la página, en el puerto 5253. Accesos de desarrollo: `?sprites` abre la galería de sprites y `?salas` va directo a la lista de salas.

12. Aplicación Android

12.1 Generar la APK

Desde la carpeta `Pacman`, definir la dirección predeterminada de la API y preparar el proyecto nativo:

```
$env:VITE_API_URL = 'http://IP-DE-TU-PC:5253'
npm run build -- --mode android
npm run android:sync
npm run android:open
```

1. En Android Studio, seleccionar la variante **debug**.
2. Usar **Build → Build APK(s)**.
3. La APK queda en `Pacman/android/app/build/outputs/apk/debug/app-debug.apk`.
4. Copiarla al celular e instalarla (permitir la instalación desde esa fuente si Android lo pide).

Como alternativa a la variable de entorno se puede copiar `.env.android.example` a `.env.android.local` y escribir allí `VITE_API_URL`. Ese archivo está ignorado por Git.

> Después de cambiar código del cliente siempre se debe volver a ejecutar `npm run build -- --mode android` y `npm run android:sync`; si no, la APK sigue con la versión anterior.

12.2 Conectar el celular al servidor

La dirección de compilación es solo la predeterminada; se puede cambiar desde la app en **SERVIDOR** sin generar otra APK. Hay dos formas:

Misma red Wi-Fi (IP de la PC)

1. Iniciar `PacmanApi` con el perfil **red-local**.
2. Conectar la PC y el celular a la misma Wi-Fi (sin aislamiento entre dispositivos).
3. Ver la IP en la página `/conectar` o con `ipconfig`.
4. En la app: **SERVIDOR**, escribir la IP (por ejemplo `192.168.1.100`; el puerto 5253 se agrega solo) y **GUARDAR SERVIDOR**.

Desde cualquier red (Dev Tunnels de Visual Studio)

1. Iniciar sesión en Visual Studio y, en el menú de depuración, entrar a **Dev Tunnels → Crear un túnel**.
2. Elegir **Persistente** y acceso **Público**, y activarlo.
3. Iniciar o reiniciar `PacmanApi` con el perfil **red-local**. La página `/conectar` muestra primero la tarjeta del túnel con su QR.
4. En la app: **SERVIDOR → ESCANEAR QR**, conceder la cámara y leer el QR. También se puede escribir la URL del túnel y pulsar **GUARDAR SERVIDOR**.

La PC, Visual Studio, `PacmanApi` y SQL Server deben seguir encendidos durante la partida. **PREDETERMINADA** vuelve a la dirección incluida al compilar.

13. Solución de problemas

Problema	Causa probable	Solución
"No se pudo conectar con el servidor"	API apagada, IP incorrecta o firewall	Verificar que `PacmanApi` esté corriendo con el perfil red-local, revisar la IP en SERVIDOR y permitir el puerto TCP 5253 en el firewall de Windows
El celular no abre Swagger	Perfil http (solo localhost) o redes distintas	Usar el perfil red-local y la misma Wi-Fi, o usar el túnel
La APK no conecta pero el navegador del celular sí	Android bloquea HTTP sin cifrar	Compilar la variante debug (tiene permitido el tráfico HTTP)
No llega el correo con el código	Credenciales de Gmail sin configurar o incorrectas	Configurar `Correo:Usuario` y `Correo:Contrasena` con User Secrets; en desarrollo, buscar el código en la consola de la API
Error al iniciar la API con la base	Cadena de conexión o scripts faltantes	Revisar `ConnectionStrings:PacmanDb` y ejecutar los scripts 01 a 09
"Verificando sesión..." se queda unos segundos	La API no responde	Esperar (termina en unos 6 s) o cambiar la dirección en SERVIDOR
El túnel no aparece en /conectar	Túnel inactivo o API iniciada antes de activarlo	Activar el túnel en Visual Studio y reiniciar `PacmanApi`
La app nueva sigue mostrando el ícono anterior	Caché del launcher de Android	Desinstalar la versión anterior antes de instalar
Cambios del cliente no aparecen en la APK	Faltó compilar y sincronizar	Ejecutar `npm run build -- --mode android` y `npm run android:sync`
"Iniciaron sesión con tu cuenta en otro dispositivo"	La misma cuenta se abrió en otro equipo	Usar una cuenta por persona

14. Anexos

14.1 Glosario

Término	Significado
Anfitrión	Jugador que creó la sala; configura e inicia la partida
APK	Paquete de instalación de una aplicación Android
Capacitor	Herramienta que empaqueta una aplicación web como app nativa
DTO	Objeto de transferencia de datos entre servidor y cliente
Dev Tunnel	Servicio de Visual Studio que publica un puerto local en una URL HTTPS pública
Entity Framework Core	ORM de .NET que traduce clases de C# a tablas de SQL Server
Hub	Punto de conexión de SignalR al que los clientes llaman y desde el que reciben avisos
Interpolación	Calcular posiciones intermedias entre dos estados recibidos para dibujar un movimiento fluido
Lobby	Estado de la sala antes de iniciar, donde se unen los jugadores
Phaser	Framework de JavaScript/TypeScript para juegos 2D en el navegador
Ronda	Un nivel jugado dentro de una partida
SignalR	Biblioteca de ASP.NET Core para comunicación en tiempo real (WebSockets)
Tick	Paso fijo de simulación del servidor (50 ms)
Token	Cadena secreta que identifica la sesión del usuario
Vite	Herramienta de desarrollo y empaquetado para proyectos web

14.2 Constantes principales

Constante	Valor	Archivo
Niveles por partida	5	ConfiguracionPartida.cs
Humanos por sala	1 a 4 (máximo 2 Pac-Man y 2 fantasmas)	ConfiguracionPartida.cs
Salas simultáneas	5	GestorSalas.cs
Vidas	1 a 5 (por defecto 3)	ConfiguracionSala.cs
Reaparición de fantasmas en modo clásico	30, 20, 10 o 5 s (por defecto 10)	ConfiguracionSala.cs
Intervalo de cajas	Pocas 20 s, Normal 12 s, Muchas 6 s	ReglasJuego.cs
Distancia del teletransporte	6 casillas	ReglasJuego.cs
Vigencia de la sesión	30 días	UsuarioLN.cs
Vigencia del código	15 minutos, 5 intentos, reenvío cada 60 s	UsuarioLN.cs
Puerto de la API	5253 (HTTP)	launchSettings.json, Configuracion.ts
Puerto del cliente de desarrollo	8080	vite/config.dev.mjs

14.3 Referencias

- Documentación de ASP.NET Core y SignalR: learn.microsoft.com/aspnet/core
- Entity Framework Core: learn.microsoft.com/ef/core
- Phaser 4: phaser.io y docs.phaser.io
- Capacitor: capacitorjs.com/docs
- Vite: vite.dev
- Dev Tunnels: learn.microsoft.com/azure/developer/dev-tunnels